

JSIntel

JavaScript Asset Intelligence & Recon Framework

System Architecture & UML Component Reference

REPOSITORY	github.com/maxamin/jsintel
AUTHOR / MAINTAINER	maxamin
PROJECT STATUS	Phase 1 skeleton — Phase 2 plugin migration in progress
SCOPE	Authorized security testing and client-side asset inventory
DOCUMENT VERSION	1.0

1. Project Overview

JSIntel is a JavaScript Asset Intelligence and Recon Framework built for authorized security testing and client-side asset inventory. Given a supplied scope of seed URLs, it crawls the target, identifies client-side assets (JavaScript, source maps, JSON, WebAssembly, workers, and manifests), downloads them with integrity metadata, extracts inventory-relevant signals, and produces SQLite-backed reports in JSON, Markdown, and CSV formats.

The project is currently published as a **Phase 1 skeleton**: the core `jsintel.sh` pipeline is functional, while Phase 2 introduces an additive migration to a typed, plugin-based intelligence platform. The legacy shell workflow remains compatible while new collection, analysis, graph, and reporting capabilities move behind stable plugin interfaces.

1.1 Key Features

- Katana-based, JavaScript-aware crawling for .js, .mjs, source maps, JSON, WebAssembly, workers, and manifests.
- URL normalization and type classification ahead of parallel, resumable downloads.
- Integrity metadata per asset: SHA-256 hash, byte size, MIME type, failure status, timeout and retry handling.
- Extraction of URLs, API/GraphQL-style paths, WebSocket endpoints, imports/requires, and framework signatures.
- SQLite database with assets, URLs, endpoints, technologies, and findings tables.
- JSON, Markdown, and CSV reporting, plus a streaming Python extraction engine with typed findings.

1.2 Intended Use

JSIntel is designed to be pointed only at scopes the operator is explicitly authorized to assess. The crawler falls back to recording supplied HTTP(S) URLs as candidate assets when Katana is unavailable, which supports offline or constrained environments without performing live crawling.

2. System Architecture

The diagram below presents the JSIntel component architecture as ten cooperating layers: the CLI entry point, the collection layer (crawling and source adapters), the asset management pipeline, a planned AI deobfuscation and semantic layer, the plugin-based analysis engine, runtime intelligence via browser instrumentation, the SQLite knowledge layer, the correlation and risk-scoring intelligence layer, the reporting layer, and supporting external systems. Components shown with dashed borders are planned and not yet part of the Phase 1 skeleton.

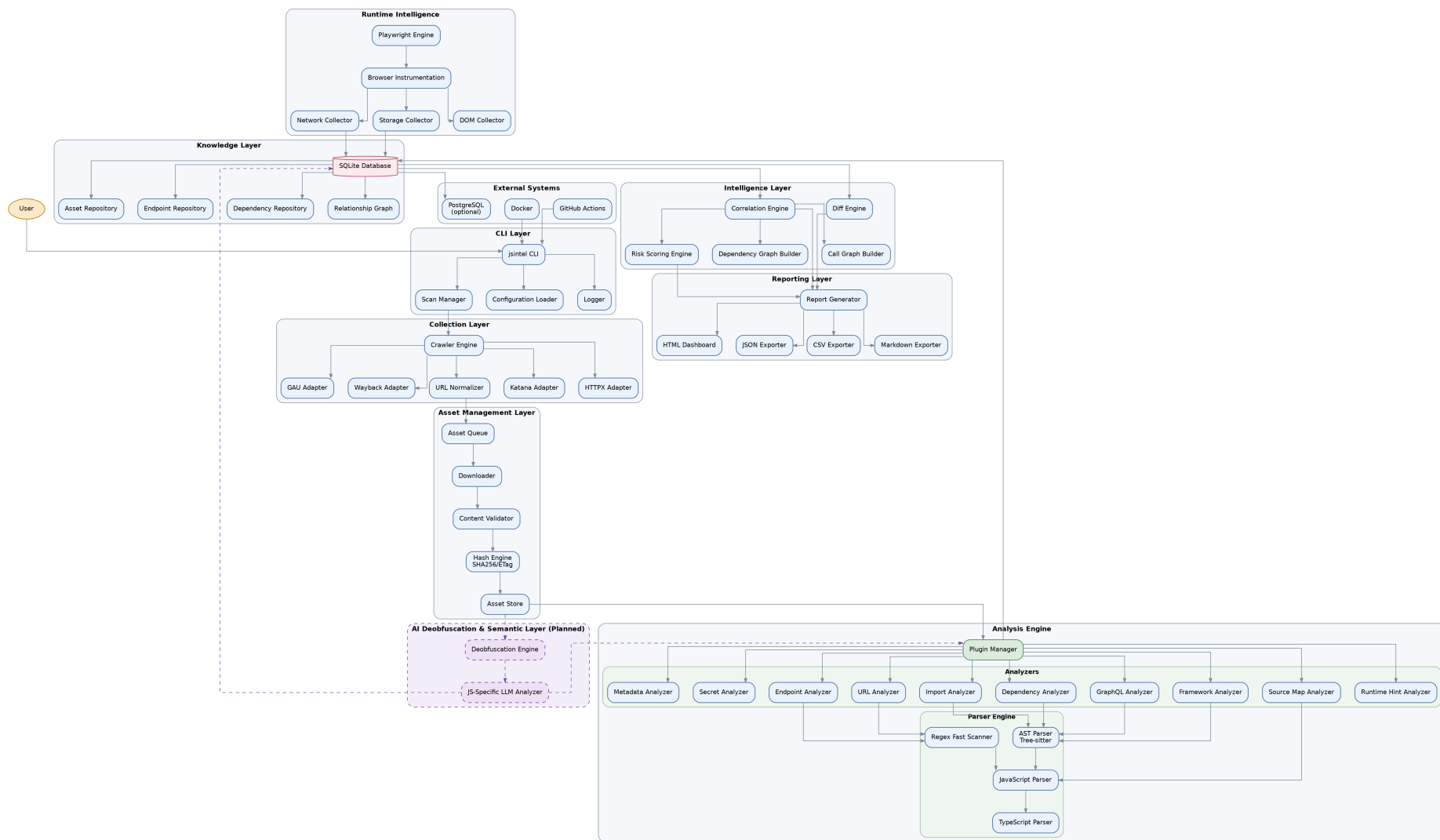


Figure 1. JSIntel component architecture (UML component diagram).

3. Layer Descriptions

CLI Layer

Entry point for operators. The jsintel CLI loads configuration via the Configuration Loader, hands off scan orchestration to the Scan Manager, and routes structured output through the Logger.

Collection Layer

The Crawler Engine coordinates source adapters — Katana, HTTPX, GAU, and Wayback — to discover candidate URLs, which the URL Normalizer deduplicates and classifies before queuing.

Asset Management Layer

The Asset Queue feeds a resumable Downloader. Each fetched artifact passes through the Content Validator and Hash Engine (SHA-256 / ETag) before being persisted to the Asset Store with integrity metadata.

AI Deobfuscation & Semantic Layer (Planned)

Before static analysis, every asset in the Asset Store is intended to pass through a Deobfuscation Engine that normalizes minified, packed, and obfuscated JavaScript back into readable form. The deobfuscated tree is then handed to a JavaScript-specific LLM Analyzer, which reads the full asset tree — not just individual files in isolation — to surface semantic findings that pattern-based analyzers miss, such as intent, hidden logic, and cross-file relationships. Results feed both the knowledge layer and the downstream analyzer plugins, without displacing them.

Analysis Engine

A Plugin Manager dispatches stored assets to ten independently discovered analyzer plugins (URL, Endpoint, Import, Framework, GraphQL, Dependency, Source Map, Secret, Metadata, and Runtime Hint). Analyzers use a shared Parser Engine — a fast Regex scanner and a Tree-sitter based AST parser — backed by JavaScript and TypeScript parsers.

Runtime Intelligence

An optional Playwright-driven Browser Instrumentation layer captures live Network, Storage, and DOM signals for targets that require rendered-page analysis, feeding results back into the knowledge layer.

Knowledge Layer

A central SQLite database is the system's source of truth, exposing repository interfaces for assets, endpoints, and dependencies, plus a relationship graph linking discovered entities. An optional PostgreSQL backend is supported for larger deployments.

Intelligence Layer

The Correlation Engine builds dependency and call graphs from stored findings and feeds a Risk Scoring Engine. A Diff Engine compares scan runs over time to surface changes.

Reporting Layer

The Report Generator renders correlation, risk, and diff output through JSON, CSV, and Markdown exporters, plus an HTML Dashboard with an asset explorer, endpoint explorer, dependency graph, timeline, technology inventory, and runtime map.

External Systems

GitHub Actions and Docker drive the CLI in CI/CD and containerized contexts; the knowledge layer can persist to PostgreSQL when configured for shared or long-term storage.

4. Plugin Contract

All intelligence modules in the Analysis Engine are implemented as plugins. Every analyzer plugin exposes a consistent four-method contract, allowing new analyzers to be added without changes to the Plugin Manager or downstream reporting:

- **initialize()** — prepare plugin state and load configuration.
- **analyze()** — process a given asset and emit typed findings.
- **finalize()** — flush buffered results and release resources.
- **schema()** — declare the structure of findings the plugin produces.

5. Knowledge Layer Data Model

The SQLite database is the system of record for a scan. It stores assets, URLs, endpoints, dependencies, detected frameworks, JavaScript chunks, captured runtime requests, analyzer findings, and scan history, giving the Intelligence and Reporting layers a single consistent view of the target.

6. Roadmap: AI-Powered Deobfuscation & Semantic Analysis

A planned enhancement to the pipeline routes the entire downloaded asset tree through a deobfuscation stage before it reaches the analysis engine, followed by a pass from a JavaScript-specific large language model rather than pattern-matching alone.

- **Deobfuscation Engine** — runs after assets are hashed and stored, unpacking minified, control-flow-flattened, or otherwise obfuscated JavaScript into a normalized, readable form across the full asset tree.
- **JS-Specific LLM Analyzer** — consumes the deobfuscated tree as a whole (not file-by-file) to reason about cross-file structure, intent, and hidden behavior that regex- and AST-based analyzers are not designed to catch.

- **Integration point** — findings from this stage are written back to the SQLite knowledge layer and made available to the existing analyzer plugins, complementing rather than replacing the Phase 1 analysis pipeline.

This stage is represented in Figure 1 with dashed borders to indicate it is on the roadmap and not yet part of the current Phase 1 skeleton.

7. Installation & Usage

On Kali Linux or Debian:

```
chmod +x install.sh jsintel.sh modules/*.sh modules/*.py
./install.sh
```

The installer places Katana and HTTPX under the Go bin path. Playwright's browser runtime is only required if extending the project with browser automation — it is not needed for the Phase 1 Katana crawler.

7.1 Running a Scan

Create a scope file with one authorized seed URL per line, then run the pipeline:

```
./jsintel.sh -i crawled_urls -o target_output -t 50
```

The `-o` flag defaults to `./output`; `-t` reads thread/timeout defaults from `config/config.yaml` (50). A single URL may also be passed directly to `-i`.

7.2 Output Layout

```
target_output/
  assets/           # crawled_urls.txt, downloaded assets
  database/recon.db
  logs/jsintel.log
  reports/          # assets.json, urls.json, endpoints.json,
                    # websocket.json, imports.json, frameworks.json,
                    # assets.csv, summary.json, summary.md
```

Ad-hoc queries are available through the database module, for example: `python3 modules/database.py --output target_output query 'SELECT endpoint FROM endpoints'`.

7.3 Project Layout

```
jsintel-phase1/
  jsintel.sh        # Pipeline launcher
  install.sh        # Debian/Kali dependency installation
  modules/          # Crawl, classify, download, extract, database, report
  database/schema.sql # SQLite schema
  config/config.yaml # Runtime defaults
  reports/          # Checked-in report exports
  output/           # Default runtime output root
```

8. Operational Notes

- The crawler uses Katana when it is installed; if unavailable, it records the supplied HTTP(S) URLs as candidate assets rather than crawling pages — useful for offline or constrained environments.
- Downloads are limited to discovered HTTP(S) URLs and are saved using sanitized filenames.
- One malformed or missing asset no longer aborts a full scan; errors are captured in `errors.json`.
- JSIntel should never be run outside a scope the operator is explicitly authorized to assess.